



PROGRAMAÇÃO PARA A INTERNET II

Caderno de Roteiros Práticos da Unidade Curricular

Curso Superior de Tecnologia em Sistemas para a Internet — IFSC Garopaba

Sumário

1	Visão Geral da Unidade Curricular	2
2	Roteiro Prático 01 — Retomada de Conteúdos: DOM, Assincronismo e Persistência Local	4
2.1	Introdução	4
2.2	Parte 1 — Revisão Conceitual e Estrutura Inicial (30 minutos)	4
2.3	Parte 2 — Consulta Assíncrona de CEP com Fetch API (35 minutos)	5
2.4	Parte 3 — Manipulação do DOM e Persistência com LocalStorage (35 minutos)	5
2.5	Checklist Final da Aula	6



1 Visão Geral da Unidade Curricular

A Unidade Curricular de **Programação para a Internet II** tem como objetivo aprofundar os conhecimentos adquiridos na disciplina antecedente (Programação para a Internet I), evoluindo o desenvolvimento frontend básico em HTML5, CSS3 e JavaScript Vanilla para a construção de **Aplicações Web Modernas, Reativas, Modulares e Integradas a Serviços Web RESTful**.

Ao longo do semestre, os estudantes irão desenvolver competências avançadas em arquitetura de código client-side, modularização com ES6 Modules, manipulação avançada de estado no cliente, consumo de APIs assíncronas, roteamento no cliente (Single Page Applications — SPAs) e introdução a ferramentas modernas de build e bibliotecas de interface (Vite e React/Vue).

Estrutura do Semestre

O semestre está organizado em 5 módulos estruturantes de aprendizagem prática:

1. Retomada e Diagnóstico (Revisão de PI-I)

Revisão prática de manipulação do DOM, eventos, assincronismo com `async/await`, Fetch API e armazenamento com `localStorage`.

2. Arquitetura Modular e ES6 Modules

Organização profissional de código frontend, separação de responsabilidades em serviços, componentes e manipuladores de eventos.

3. Gestão Avançada de Estado e Formulários Reativos

Persistência no navegador, validações avançadas no cliente, tratamento rigoroso de erros e experiência do usuário (UX).

4. Consumo de APIs RESTful e Desenvolvimento de SPAs

Construção de Single Page Applications com roteamento client-side sem recarregamento de página e integração com rotas de backend.

5. Introdução a Ecossistemas Modernos e Projeto Final

Uso de bundlers (Vite), introdução à componentização reativa e entrega de um projeto Web completo de final de semestre.

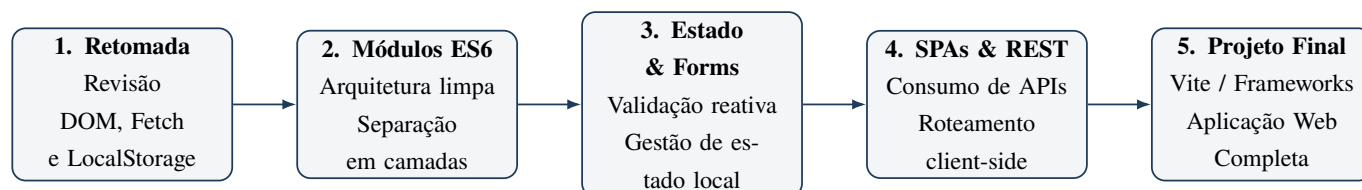


Figura 1: Pipeline de aprendizagem da UC Programação para a Internet II.



Competências Desenvolvidas

Ao final da disciplina, o estudante deverá ser capaz de:

- Construir interfaces web altamente dinâmicas e responsivas;
- Estruturar aplicações JavaScript utilizando arquitetura modular em ES6 Modules;
- Consumir e manipular dados de serviços Web RESTful usando `fetch` e `async/await`;
- Gerenciar estado local e persistência no cliente via Web Storage API;
- Implementar padrões de projeto de Single Page Application (SPA);
- Utilizar ferramentas modernas de ambiente de desenvolvimento web (Node.js, npm, Vite);
- Publicar e versionar aplicações web profissionais no GitHub e GitHub Pages.

Metodologia de Avaliação

A avaliação será continuada e baseada na entrega de roteiros práticos semanais, avaliações de desempenho prático em laboratório e no desenvolvimento de um Projeto Web Integrador entregue e apresentado ao final do semestre.



2 Roteiro Prático 01 — Retomada de Conteúdos: DOM, Assincronismo e Persistência Local

Objetivos

Objetivos da Aula

Ao final desta aula o estudante será capaz de:

- Reativar os conceitos fundamentais de HTML5 semântico e CSS3 (Flexbox/Grid);
- Manipular a árvore DOM e gerenciar eventos de formulário dinamicamente com JavaScript ES6+;
- Realizar requisições assíncronas a APIs REST públicas utilizando `fetch` e `async/await`;
- Persistir, recuperar e remover dados no navegador utilizando a `localStorage` API;
- Desenvolver uma aplicação web completa de gestão e consulta integrada.

Introdução

Bem-vindos à disciplina de **Programação para a Internet II**!

Neste primeiro encontro, faremos uma revisão prática intensiva das competências trabalhadas no semestre anterior (PI-I). O objetivo é garantir que toda a turma domine com segurança a tríade essencial do desenvolvimento web frontend client-side: **HTML5 Semântico**, **CSS3 Responsivo** e **JavaScript Assíncrono (DOM + Fetch + LocalStorage)**.

Para este exercício inicial de retomada, vocês irão construir o **Dashboard de Cadastro de Clientes e Consulta Integrada de Endereços**, localizado na pasta `exercicio-retomada/`.

Parte 1 — Revisão Conceitual e Estrutura Inicial (30 minutos)

Atividade 1 – Organização do Projeto e Interface HTML5

Cada estudante deverá criar/acessar a pasta `exercicio-retomada/` e estruturar a interface base da aplicação:

1. Criar a estrutura de arquivos: `index.html`, `styles.css` e `app.js`;
2. No arquivo `index.html`, estruturar um documento semântico contendo:
 - Cabeçalho institucional (`<header>`);



- Formulário de cadastro de cliente (`<form id="form-cliente">`) com campos para Nome, E-mail, CEP, Logradouro, Bairro, Cidade e UF;
 - Tabela de listagem dinâmica (`<table id="tabela-clientes">`);
3. Vincular corretamente o arquivo CSS externo e a tag `<script src="app.js" defer></script>`.

Parte 2 — Consulta Assíncrona de CEP com Fetch API (35 minutos)

Atividade 2 – Integração com a API ViaCEP

No arquivo `app.js`, implementar a função assíncrona para buscar o endereço automaticamente a partir do CEP digitado:

1. Adicionar um ouvinte de evento (`addEventListener`) no campo de CEP para o evento `blur` ou `input`;
2. Criar uma função `async function buscarCep(cep)` que consulte o serviço ViaCEP:

`https://viacep.com.br/ws/{CEP}/json/`

3. Utilizar `try/catch` para tratar erros de rede ou CEPs inexistentes;
4. Preencher automaticamente os campos de Logradouro, Bairro, Cidade e UF no formulário com a resposta JSON retornado.

Parte 3 — Manipulação do DOM e Persistência com LocalStorage (35 minutos)

Atividade 3 – Cadastro, Renderização e Persistência

Implementar a lógica de cadastro e listagem dinâmica no `app.js`:

1. Capturar o evento `submit` do formulário (`e.preventDefault()`);
2. Montar um objeto JavaScript com os dados do cliente e salvá-lo em um array global no `localStorage` (`JSON.stringify`);
3. Criar a função `renderizarTabela()` para atualizar a árvore DOM criando linhas (`<tr>`) e colunas (`<td>`) dinamicamente;



4. Implementar o botão de exclusão para cada cliente cadastrado, atualizando o `localStorage` e o DOM em tempo real;
5. Garantir que, ao recarregar a página, a função de carregamento inicial leia o `localStorage` (`JSON.parse`) e exiba os dados persistidos.

Checklist Final da Aula

Verificação Técnica

Antes de finalizar o encontro, verifique se seu projeto atende aos critérios de aceitação:

- A página possui marcação HTML5 semântica e estilização CSS responsiva em Flexbox/Grid;
- O preenchimento do CEP realiza requisição `fetch` e autocompleta os campos de endereço sem recarregar a página;
- O cadastro adiciona o novo cliente na tabela dinamicamente via manipulação do DOM;
- Os dados persistem no `localStorage` mesmo após fechar ou recarregar o navegador;
- A remoção de registros funciona corretamente atualizando o estado persistido.

Reflexão Técnica

Aplicações web modernas fundamentam-se no domínio do assincronismo e na gestão de estado client-side. Dominar a manipulação limpa do DOM, o tratamento de respostas HTTP JSON e a persistência local é o requisito básico para avançarmos em direção aos frameworks reativos (React, Vue) e arquiteturas de Single Page Application (SPA) neste semestre!